AIORI HACKATHON 3

Authorised Attack-Surface Inventory and Change Monitor

PS-011 | Technical Report

TEAM CODE	TEAM NAME
A3-PS011-TC064	Cyber Warriors
SUBMITTER	**INSTITUTE**
Daksh Jain	GLS University
REPORT DATE	**REPOSITORY**
17 October 2026	aiori-hackathon/Cyber-Warriors

Scope and evidence statement

This report describes the prototype and its seeded demonstration dataset. The reported findings are not independently verified vulnerabilities on real external systems. All active checks must remain within the written, organiser-approved scope.

Prepared against the supplied Proposed-structure-hackathon.pdf section order.

Abstract

Cyber Warriors is developing a bounded attack-surface inventory and change-monitoring prototype for the AIORI Hackathon 3 PS-011 challenge. The system is intended to track approved lab domains and hosts, collect DNS and TLS observations, inspect selected TCP ports, compare current observations with a saved baseline, and present changes through a dashboard and report outputs.

The prototype includes four documented inspection layers: DNS records and potential dangling-CNAME indicators; TLS certificate validity and expiry; selected-port/service observations; and baseline comparison with finding lifecycle support. Seeded scenarios provide repeatable examples for a mentor demonstration.

Evidence boundary: recent project notes report 18 passing unit tests and 4 passing scenario/demo tests. Those counts are reported project evidence, not a fresh independent test run performed while preparing this PDF. The dashboard report snapshot contains seven seeded findings and ten recorded change events; these counts describe the demonstration dataset, not independently confirmed external vulnerabilities.

Problem Description

Organisations can lose track of exposed assets as test systems are retired, certificates expire, or services change. A DNS record may continue to point at a decommissioned provider resource, and a new service may appear outside the expected baseline. Without repeatable inventory and review, changes can be missed or interpreted inconsistently.

- Maintain a bounded inventory of up to 20 lab domains and 10 lab hosts.
- Perform the four documented checks against only organiser-approved targets.
- Compare observations over time and surface changes with evidence for human review.
- Use seeded test scenarios to demonstrate expired certificates, unexpected services and dangling-DNS candidates.
- Document uncertainty and false-positive risk; do not claim a candidate is exploitable without independent validation.

Background

An attack-surface inventory records assets and observable properties that require security oversight. It becomes more useful when the same approved assets are checked repeatedly and observations are compared against a known baseline. DNS lookups show how names resolve, TLS inspection records certificate information, and selected TCP connection checks provide a limited view of reachable services. Baseline comparison adds time context to these observations.

Operational principles

- **Authorised scope:** Use only written, organiser-approved lab domains and hosts. Validate scope before network checks.
- **Bounded inspection:** Query documented DNS record types and inspect only configured TCP ports; avoid exploitation or unrestricted scanning.
- **Evidence-first findings:** Preserve timestamps, observations and the reason a rule fired. A timeout or generic HTTP 404 is not automatically a vulnerability.
- **Human review:** Keep uncertain findings open for review and support false-positive or resolution decisions where the application provides those states.
- **Reproducibility:** Use seeded/synthetic scenarios for repeatable demonstrations and retain test outputs that can be re-run.

Interpretation and limitations

Provider-specific error pages and multi-tenant cloud endpoints can produce ambiguous results. Potential dangling DNS indicators should be treated as candidates. A service identified as unexpected should be compared with the intended baseline and confirmed with the system owner. Certificate findings should be interpreted from certificate data and connection context, not from a failed connection alone.

The supplied audit draft also recorded implementation caveats around settings propagation and an asset-edit workflow. This report does not assert those caveats are fixed unless verified by current code and regression evidence; they should be checked during the final demonstration.

Literature & RFC Review

The following standards and documentation are relevant to the project's bounded DNS, TLS, TCP and web-application checks. References provide technical context; they do not by themselves prove that every protocol property is implemented or tested.

Reference	Relevance	Use and limitation
OWASP Web Security Testing Guide (WSTG)	Structured guidance for authorised security testing.	Use as methodology context; keep checks within approved scope.
IETF RFC 1035 — Domain Names	DNS concepts and record lookup behaviour.	Relevant to A, AAAA, CNAME and MX observations.
IETF RFC 8446 — TLS 1.3	TLS protocol context.	Certificate parsing does not imply complete TLS protocol auditing.
Python socket documentation	TCP connection attempts and timeout behaviour.	Interpret connection results cautiously; timeout is not proof of closed port.
dnspython documentation	DNS resolver API and query handling.	Document configured query types and resolver-error handling.
pyca/cryptography documentation	X.509 certificate parsing and fields.	State which certificate attributes and expiry calculations are used.
Flask documentation	Web routes and application testing.	Route health tests do not prove every workflow is correct.

References to verify before submission

OWASP WSTG: https://owasp.org/www-project-web-security-testing-guide/
RFC 1035: https://www.rfc-editor.org/rfc/rfc1035
RFC 8446: https://www.rfc-editor.org/rfc/rfc8446
Python socket: https://docs.python.org/3/library/socket.html
dnspython: https://dnspython.readthedocs.io/
cryptography: https://cryptography.io/
Flask: https://flask.palletsprojects.com/

Submission note: retain references the team actually consulted and confirm access dates if required by the organiser.

Proposed Solution

The prototype is described as a Python application with a Flask dashboard, SQLite-backed records, inspection modules, baseline/change detection, seeded scenarios and report-generation features. The implementation should be demonstrated using only the approved lab asset list and controlled test fixtures.

Four documented checks

Check	Purpose	Caution
1. DNS records and dangling-CNAME candidates	Collect A, AAAA, CNAME and MX observations; identify provider-specific signatures that may indicate an orphaned destination.	A signature or HTTP 404 is a candidate signal, not proof of takeover.
2. TLS certificate verification	Inspect certificate data and expiry; flag expired certificates.	A failed handshake alone does not establish certificate expiry.
3. Selected TCP ports and services	Check configured ports and record available service/banner evidence.	Only use approved hosts and the configured port allowlist.
4. Baseline comparison and change detection	Compare current observations with a saved baseline and produce change events/findings.	Confirm baseline correctness and review noisy or duplicate findings.

Findings and review

The dashboard/report snapshot used for this report contains seven seeded findings: three potential dangling-DNS indicators, one expired-certificate scenario, and three unexpected-service scenarios. The same snapshot records ten change events. These values may change when the database is reset or a new demonstration scan is run. Findings should be reviewed with supporting evidence and should never be represented as confirmed real-world vulnerabilities solely because a seeded rule fired.

The prototype’s purpose is inventory and change awareness, not exploitation. No takeover attempts or unrestricted scans are part of the intended workflow.

Solution Optimization

The main optimisation is a repeatable workflow: a bounded scope, documented checks, stored observations, baseline comparison, and a reviewable report. No controlled performance benchmark was provided, so this report does not claim a specific percentage improvement in scan speed or detection accuracy.

Area	Before / risk	Targeted improvement	Acceptance evidence
Asset inventory	Manual or inconsistent lists can drift.	Keep a bounded, explicit approved asset list.	Scope rejects an unapproved target; inventory stays within project limits.
DNS/TLS/service checks	Separate checks can be difficult to compare.	Use four documented checks and consistent observation records.	Repeatable seeded scenarios create expected categories of observations.
Change tracking	A single scan does not show what changed.	Compare observations with a stored baseline and record events.	Known baseline changes appear in the report with timestamps.
False-positive handling	Provider error pages and timeouts may be misread.	Show confidence/FP risk and retain a human-review step.	CloudFront-style ambiguous response remains a candidate, not a confirmed issue.
Reporting and demo	Evidence may be spread across terminal output and pages.	Provide a dashboard, screenshots, and a structured report.	Mentor can follow a repeatable demo and see scope/evidence caveats.

Verification status

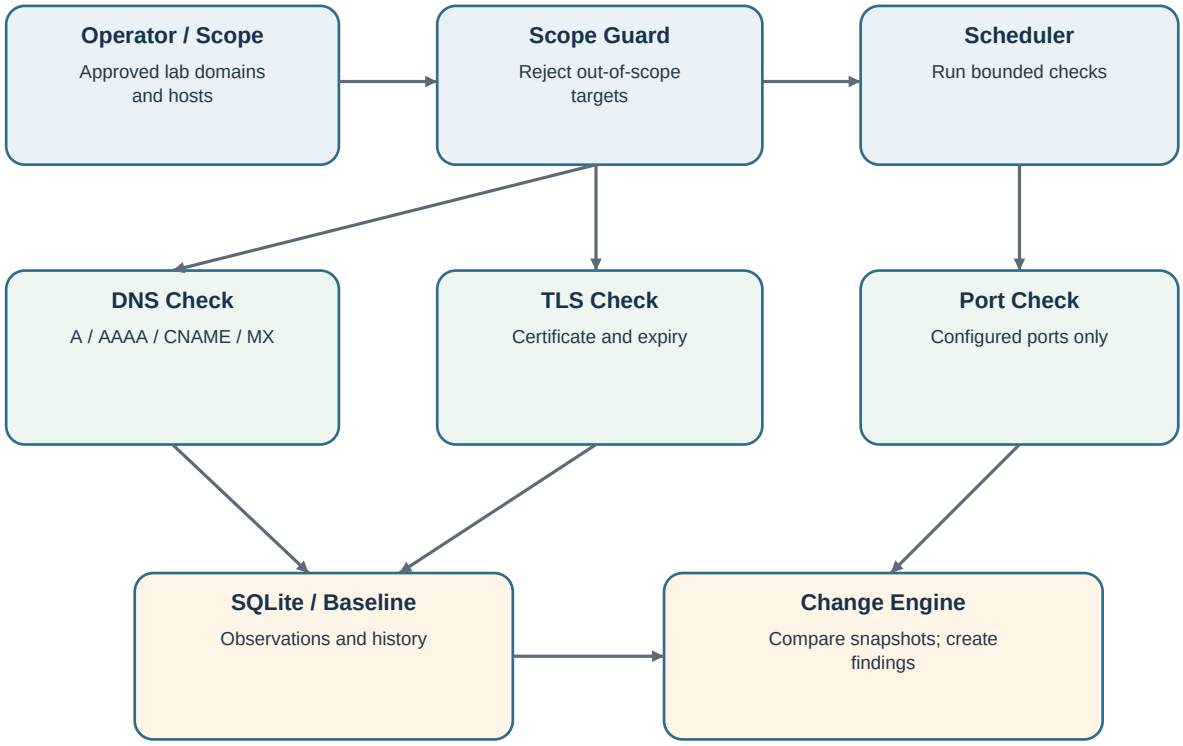
Recent project notes report 18 passing unit tests and 4 passing scenario/demo tests. The latest GitHub push completed at commit **93e8553**. These are recorded project results; the PDF authoring process did not rerun the tests or independently inspect the current repository source. Before submission, run both test commands from the project directory and retain their actual output.

Suggested regression checks

- Run `python -m unittest discover tests`.
- Run `python main.py test`.
- Start the dashboard using the verified project startup command and confirm the required pages load.
- Verify settings propagation, unexpected-service deduplication, and any asset-edit workflow if those remain in the current acceptance criteria.

Solution Architecture & Design

The component diagram below summarises the intended logical flow from approved assets to bounded inspection, storage, change analysis, and review. It is a conceptual diagram based on the project description, not a code-generated UML model. Confirm exact module names and connections against the final source revision.



Logical component view — confirm module mapping against the final source revision.

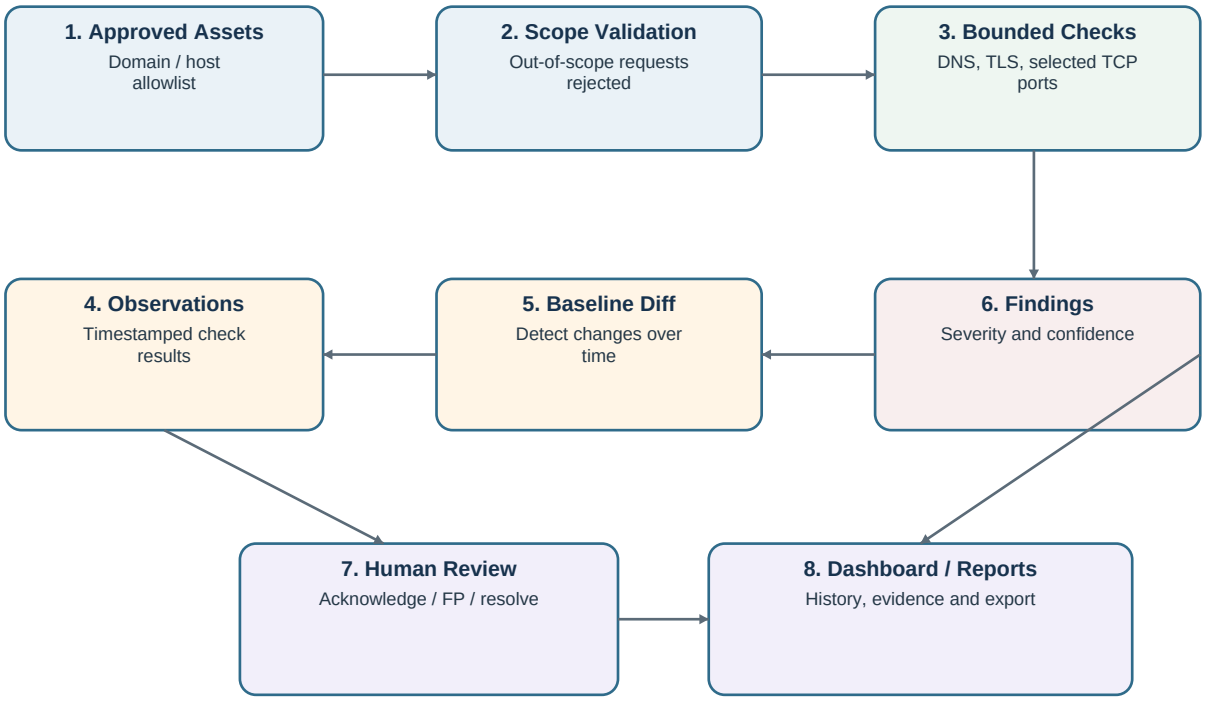
Architecture responsibilities

- **Scope and orchestration:** choose approved domains/hosts and schedule the documented checks.
- **Inspection modules:** gather DNS, certificate, and selected-port observations.
- **Storage and change engine:** persist observations, compare baselines, and create findings/events.
- **Dashboard and reports:** expose history and evidence for human review.

The diagram does not assert that every legacy module or entry point is active. Use the same verified startup path during the demonstration.

Data Flow Diagram

This data-flow diagram illustrates the intended sequence for a controlled lab scan. Scope validation precedes inspection, results are stored and compared with a baseline, and findings are presented for review.



Illustrative data flow for the monitored lab workflow.

Flow explanation

- An operator selects assets that appear on the written, organiser-approved scope list.
- The scope guard validates requested targets; out-of-scope requests should be rejected.
- DNS, TLS and selected-port checks collect bounded observations.
- Observations are stored and compared with the previous baseline.
- The change engine creates candidate findings and timestamped change events.
- The dashboard and report expose evidence; a human decides whether to acknowledge, resolve, or mark a result as a false positive.

Use synthetic/seeded scenarios in the video and mentor demo. Do not test public targets unless they are explicitly authorised in writing.

Deployment Timeline

The milestones below describe a practical release and demonstration sequence. Statuses should be updated by the team from direct evidence before the final portal submission.

Milestone	Activities	Exit evidence / status
1. Repository and scope	Confirm private repository, organiser access, approved assets and configured port list.	Repository push verified at commit 93e8553; organiser access/transfer and portal state must be checked separately.
2. Run regression tests	Execute unit and scenario test commands after the final code changes.	Latest project notes report 18 unit tests and 4 scenario tests passing; rerun and preserve exact output.
3. Dashboard rehearsal	Start the app from a fresh terminal; show assets, check results, change/finding review and reports.	Capture screenshots and ensure only approved or synthetic data is shown.
4. Report and presentation	Review the technical report, citations, architecture/DFD diagrams and presentation PDF.	Confirm all facts match current code and evidence; do not present seeded findings as verified external vulnerabilities.
5. Team video	Record team introductions, problem statement, solution, dashboard demo, tests and responsible-use note.	Upload to YouTube and add the link to the repository if required by the organiser.
6. Final submission	Check the official submission portal, repository access requirements, PDF and demonstration deliverables.	Team confirms every required deliverable is uploaded and portal status is recorded.

Demonstration command sequence

```
python -m unittest discover tests
python main.py test
python app.py
```

Open the local dashboard URL shown by the application. Earlier project notes referenced ports 5005 and 5000 at different points; use the port printed by the current startup command rather than assuming a fixed port.

Bibliography

[1] OWASP Foundation. *Web Security Testing Guide (WSTG)*. https://owasp.org/www-project-web-security-testing-guide/

[2] Mockapetris, P. *RFC 1035: Domain Names — Implementation and Specification.* Internet Engineering Task Force. https://www.rfc-editor.org/rfc/rfc1035

[3] Rescorla, E. *RFC 8446: The Transport Layer Security (TLS) Protocol Version 1.3.* Internet Engineering Task Force. https://www.rfc-editor.org/rfc/rfc8446

[4] Python Software Foundation. *socket — Low-level networking interface.* https://docs.python.org/3/library/socket.html

[5] dnspython contributors. *dnspython documentation.* https://dnspython.readthedocs.io/

[6] pyca/cryptography contributors. *Cryptography documentation.* https://cryptography.io/

[7] Pallets Projects. *Flask documentation.* https://flask.palletsprojects.com/

[8] AIORI Hackathon 3. *PS-011: Authorised Attack-Surface Inventory and Change Monitor.* Supplied problem statement PDF.

[9] AIORI Hackathon 3. *Proposed-structure-hackathon.pdf.* Supplied report template.

Evidence and integrity notes

- Test counts and the latest commit identifier in this report come from project notes supplied in the working conversation; they were not independently rerun or checked against GitHub during PDF generation.
- The finding counts refer to a seeded demonstration dataset. They are not a claim of verified external vulnerabilities.
- Confirm the exact sources consulted and any required access dates before submission.
- The official problem statement requires authorised scope, four documented checks, scheduled comparisons, seeded issue detection, false-positive reporting, a final technical report, and a demonstration.

Thank You for Your Attention!

Cyber Warriors | A3-PS011-TC064

PS-011: Authorised Attack-Surface Inventory and Change Monitor

Responsible-use commitment

This prototype is intended for written, organiser-approved lab scope only. It performs bounded inspection and change monitoring; it does not attempt exploitation. A potential dangling DNS result is a candidate for review, not proof of takeover. Seeded findings must be described as test scenarios unless independently verified in an authorised environment.

Before final submission, verify the current code, rerun tests, check the repository and organiser-access requirements, upload the required video if applicable, and confirm the submission portal status.

Cyber Warriors • GLS University • AIORI Hackathon 3